

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 13: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions and data types live in `starter_code.py` at the week root: `generate_graph`, `generate_tree`, `edges_to_adj`, `is_vertex_cover`, `is_independent_set`, `brute_force_vertex_cover`, `brute_force_independent_set`, the QBF node classes (`Const`, `Var`, `Not`, `And`, `Or`, `Quant`), `cnf_is_satisfied`, and `generate_2sat`.

The 12 problems are grouped into three themes:

- **PSPACE & Games** (Problems 1–3): recursive evaluation of quantified boolean formulas (the canonical PSPACE-complete problem), minimax game-tree evaluation, and alpha-beta pruning.
- **Fixed-Parameter Tractability & Special Structure** (Problems 4–7, 10, 12): bounded-search-tree Vertex Cover, verification against brute force, kernelization, maximum independent set on trees, 2-SAT via SCCs, and FPT set cover.
- **Branch and Bound & State-Space Search** (Problems 8, 9, 11): branch-and-bound knapsack and TSP, and iterative-deepening DFS.

1 PSPACE and Games

Problem 1 – Evaluate a Fully-Quantified Boolean Formula (QBF)

Deciding a quantified boolean formula (QSAT/TQBF) is the canonical PSPACE-complete problem. Implement `eval_qbf(formula, env)` that recursively evaluates a quantified boolean formula and returns `True` or `False`. The formula is built from the node classes in `starter_code.py`: `Const`, `Var`, `Not`, `And`, `Or`, and `Quant`.

A `Quant('E', var, body)` means “ $\exists$ var. body” (true if body is true for var = `False` or var = `True`); `Quant('A', var, body)` means “ $\forall$ var. body” (true only if body is true for *both* values). Evaluate by trying both values, recursing into the body, and combining with `any(...)` for $\exists$ and `all(...)` for $\forall$.

Example. `eval_qbf(Quant("A", "x", Or(Var("x"), Not(Var("x")))))` is `True` (the tautology $\forall x. (x \vee \neg x)$), while `eval_qbf(Quant("A", "x", Var("x")))` is `False`.

Problem 2 – Minimax Game-Tree Evaluation

Implement `minimax(node, maximizing)` returning the minimax value of a game tree *for the player to move*. A game tree is a nested structure: an internal node is a non-empty list of child subtrees; a leaf is a number (the payoff from the maximizing player’s viewpoint). If `maximizing` is `True`, return the max over children (each evaluated with `maximizing=False`); otherwise return the min (children with `maximizing=True`).

Example. For the tree `[[3,5],[2,9]]` with the root maximizing: the two MIN children evaluate to 3 and 2, and the MAX root returns 3.

Problem 3 – Alpha-Beta Pruning

Implement `alphabeta(node, maximizing, alpha, beta)` returning `(value, leaves_visited)`. Carry the bounds α (best value the maximizer can guarantee) and β (best the minimizer can guarantee) down the recursion, updating them after each child and *breaking* out of the loop when $\alpha \geq \beta$.

Verify that `alphabeta` returns the *same value* as plain minimax (`minimax_count`, provided) on every test tree, while visiting *no more* leaves – and strictly fewer leaves in total across all the test trees. This demonstrates that pruning speeds up the search without changing the answer.

2 Fixed-Parameter Tractability and Special Structure

Problem 4 – Vertex Cover via Bounded-Search-Tree FPT

Implement `vertex_cover_fpt(n, edges, k)` deciding whether the graph has a vertex cover of size $\leq k$, returning the cover (a set) or `None`. Use the bounded search tree: while uncovered edges remain and the budget is positive, pick any uncovered edge (u, v) and branch on adding u (recurse with budget $k - 1$) and on adding v . The tree has depth $\leq k$ and branching factor 2, giving $O(2^k \cdot (n + m))$ time – FPT in k .

Example. On a triangle (edges $(0, 1), (1, 2), (0, 2)$), `vertex_cover_fpt(3, ..., 1)` returns `None` but `vertex_cover_fpt(3, ..., 2)` returns a valid 2-vertex cover.

Problem 5 – Verify FPT Vertex Cover vs Brute Force

Implement `verify_fpt_vertex_cover(num_trials, n, seed)`: for each of `num_trials` random graphs (`generate_graph(n, p=0.4, seed=seed + trial)`), find the minimum vertex cover size via both `brute_force_vertex_cover` (`starter_code.py`) and the FPT decider from Problem 4 (called with increasing k). Return `True` iff the sizes agree *and* the FPT threshold is exact (a cover of size `opt` exists but none of size `opt-1`), for every trial. Also implement the helper `min_vertex_cover_via_fpt(n, edges)`.

Problem 6 – Kernelization for Vertex Cover

Implement `kernelize_vertex_cover(n, edges, k)` applying two reduction rules exhaustively for the parameterized question “is there a cover of size $\leq k$?”:

Rule 1 (high degree): any vertex of degree $> k$ **MUST** be in every cover of size $\leq k$. Force it in, delete it and its edges, decrement k .

Rule 2 (isolated): a degree-0 vertex covers nothing -- drop it.

Return `(remaining_vertices, remaining_edges, k_reduced, forced)`, where `forced` is the set of vertices forced into the cover by Rule 1. The original instance is a yes-instance iff the reduced one is (with budget `k_reduced`).

Example. For a star with center 0 and 5 leaves with $k = 1$: vertex 0 has degree $5 > 1$, so it is forced; its edges vanish and the leaves become isolated and are dropped, leaving an empty instance with $k = 0$.

Problem 7 – Maximum Independent Set on a Tree via DP

Implement `max_independent_set_tree(n, edges)` returning a maximum independent set of a tree in linear time, via a post-order DP rooted at vertex 0:

```
incl[v] = 1 + sum over children c of excl[c]
excl[v] = sum over children c of max(incl[c], excl[c])
```

The optimum size is `max(incl[root], excl[root])`; reconstruct the actual set by walking down from the root (if a vertex is included, all its children are excluded). Also implement `verify_tree_mis(num_trials, n, seed)` confirming the DP size matches `brute_force_independent_set` on random trees (`generate_tree`).

Example. On the path 0–1–2–3–4 the maximum independent set has size 3 (e.g. $\{0, 2, 4\}$).

Problem 10 – 2-SAT in Polynomial Time (Implication Graph + SCC)

Although 3-SAT is NP-complete, 2-SAT is solvable in linear time. Implement `two_sat(num_vars, clauses)` returning a satisfying assignment `{var: bool}` or `None` if the 2-CNF formula is unsatisfiable. Build the implication graph (each clause $(a \vee b)$ gives $\neg a \Rightarrow b$ and $\neg b \Rightarrow a$), compute strongly connected components (Kosaraju or Tarjan), and report unsatisfiable iff some variable x has x and $\neg x$ in the same SCC. Otherwise read off an assignment from the SCC order.

Verify both directions against a brute-force satisfiability check over all 2^n assignments on random formulas (`generate_2sat`); use `cnf_is_satisfied` (`starter_code.py`) to check assignments.

Problem 12 – Set Cover via Bitmask / FPT Search

Implement `set_cover_fpt(universe, subsets, k)` returning the indices of at most k subsets whose union is `universe` (the smallest found), or `None` if no cover of size $\leq k$ exists. Use a bounded search tree parameterized by k : while elements remain uncovered and budget > 0 , pick any uncovered element and branch over the subsets containing it. Represent covered elements as a *bitmask* for fast union and coverage tests. Also implement `min_set_cover_via_fpt(universe, subsets)` (try $k = 0, 1, \dots$) and verify against `brute_force_set_cover` (provided) on small random instances.

3 Branch and Bound and State-Space Search

Problem 8 – Branch-and-Bound 0/1 Knapsack

Implement `knapsack_branch_and_bound(values, weights, capacity)` returning the maximum total value of a 0/1 knapsack via branch and bound. Sort items by value/weight ratio (descending) for a tight relaxation, explore the include/exclude binary tree, and at each node compute the *fractional* (LP-relaxation) upper bound (`fractional_bound`, provided); prune a subtree when its bound does not exceed the best value found so far.

Verify that the optimum equals the standard $O(nW)$ DP (`dp_knapsack`, provided) on a fixed instance and on 200 random instances.

Problem 9 – Branch-and-Bound TSP

Implement `tsp_branch_and_bound(dist)` returning the length of the shortest Hamiltonian cycle through all cities, starting and ending at city 0, for a symmetric distance matrix `dist`. Branch over the order in which cities are visited and prune a partial tour when `cost_so_far + lower_bound` $\geq$ the best complete tour found so far. The provided `lower_bound` adds, for the current city and each unvisited city, the cheapest edge that could still leave it – an admissible (never-overestimating) bound.

Verify that the optimum matches Held–Karp DP (`held_karp`, provided) on random instances with small n .

Problem 11 – Iterative-Deepening DFS on a State Space

Implement `iddfs(start, goal, side, max_depth)` returning a *shortest* solution path (a list of states from `start` to `goal` inclusive) for a sliding puzzle, or `None` if no solution within `max_depth` moves. A state is a flattened tuple of a `side` $\times$ `side` board with 0 as the blank; `neighbors(state, side)` (provided) returns the states reachable by one slide. Run depth-limited DFS with limit 1, 2, 3, ... until the goal is found; the first successful limit yields a shortest path. Track visited states on the current path to avoid cycles.

Example. On the 2×2 puzzle, one slide from the goal returns a path of length 2 (start, goal).

4 LeetCode Practice

After completing the problems above, practise this week's techniques (game search / minimax, FPT / bitmask search, state-space search, branch-and-bound) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Word Ladder](#) – *Hard* – State-space BFS.
- [Sliding Puzzle](#) – *Hard* – State-space search.
- [Smallest Sufficient Team](#) – *Hard* – Set cover (FPT / bitmask).
- [Stone Game](#) – *Medium* – Game / minimax DP.
- [Stone Game II](#) – *Medium* – Game / minimax DP.
- [Can I Win](#) – *Medium* – Game search + memo.
- [Cat and Mouse](#) – *Hard* – PSPACE game (BFS on states).
- [Maximum Students Taking Exam](#) – *Hard* – Bitmask FPT.